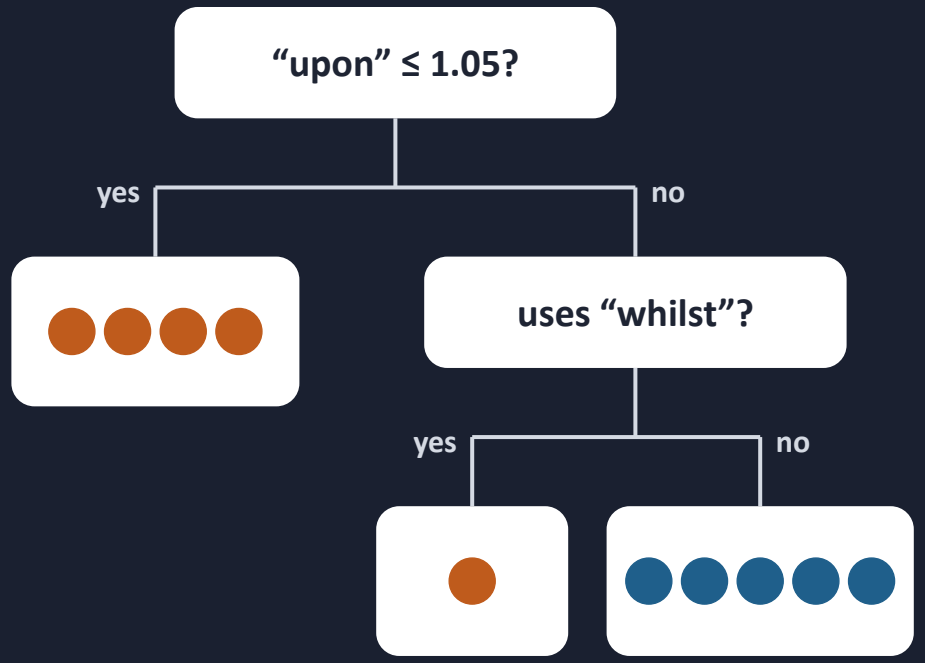
RECAP

Decision trees, step by step

How the algorithm grows a small tree, worked out with real numbers

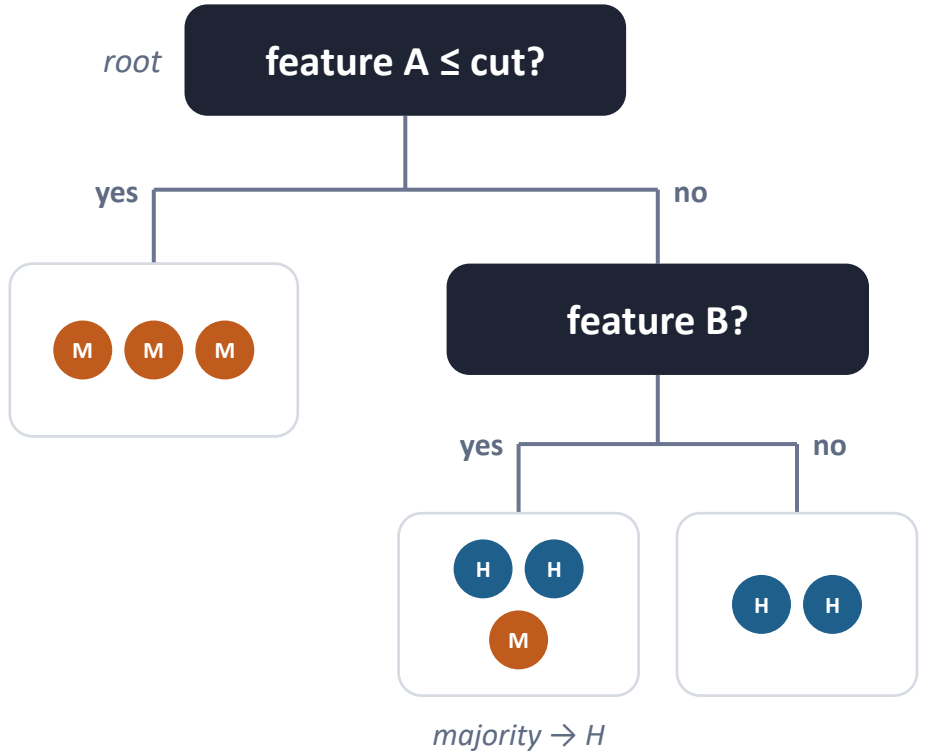


A tree is a stack of yes/no questions

Learning a tree means choosing the questions. CART, the algorithm behind scikit-learn, chooses them greedily:

1. **Start** with all the training examples in one node: the root.
2. **Try every question:** every feature and, for a numeric one, every cut between two neighbouring values.
3. **Keep the best:** score each question by how pure the two groups it creates are.
4. **Repeat inside each group** until it is pure or a limit is reached. It then becomes a leaf that predicts its majority class.

Greedy: each node takes the best split now. No look-ahead, no going back.



Question nodes split the data; leaves predict their majority class. Depth = questions on the longest path (here 2).

How mixed is a group? Gini impurity

$\text{Gini} = 1 - \Sigma\,(p_k)^2$

p_k = share of class k in the group

Read it as: the chance that two essays drawn at random from the group (with replacement) have different authors.

Scoring a split

weighted Gini = $(n_{left} / n) \times \text{Gini}_{left} + (n_{right} / n) \times \text{Gini}_{right}$

gain = Gini_{parent} − weighted Gini

Best split = lowest weighted Gini = largest gain.



0.500

$1 - 0.5^2 - 0.5^2$

maximally mixed (two classes)



0.278

$1 - (5/6)^2 - (1/6)^2$

mostly one author



0.000

$1 - 0^2 - 1^2$

pure

Alternative: entropy ($-\Sigma\, p_k \log_2 p_k$, used by ID3 and C4.5) is applied the same way and usually picks the same splits. scikit-learn's default is Gini.

Who wrote it: Hamilton or Madison?

Essay	“upon” per 1,000 words	Uses “whilst”?	Author
A	3.2	no	Hamilton
B	0.0	yes	Madison
C	2.4	no	Hamilton
D	0.3	no	Madison
E	4.0	no	Hamilton
F	2.6	yes	Madison
G	0.2	no	Madison
H	1.8	no	Hamilton
I	0.0	no	Madison
J	2.9	no	Hamilton



A classic of authorship attribution

Mosteller & Wallace (1964) attributed the 12 disputed Federalist Papers to Madison by counting function words. Hamilton wrote *upon* about 18× as often; Madison wrote *whilst* where Hamilton wrote *while*. Sentence length was useless (34.55 vs 34.59 words).

Our ten essays are invented, to keep the arithmetic small.

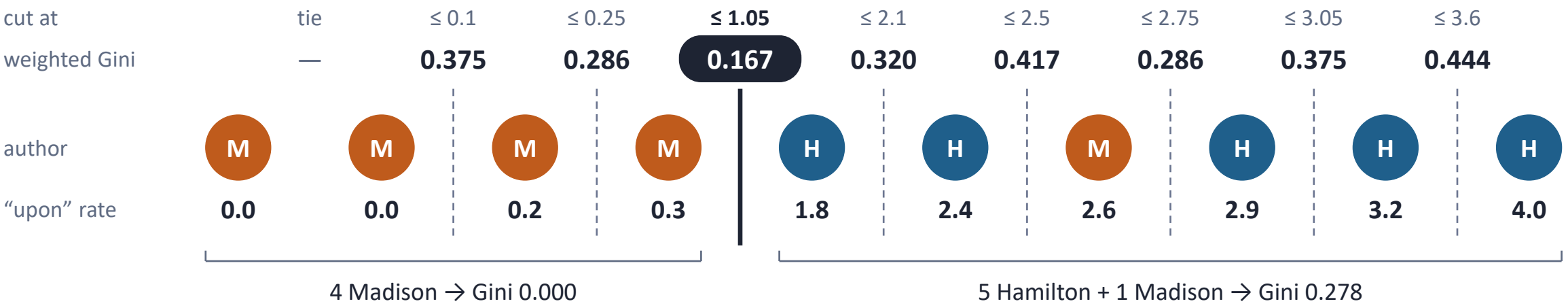
The root holds all ten essays



Gini = $1 - 0.5^2 - 0.5^2 = 0.500$

A numeric feature: try a cut in every gap

Sort the essays by “upon” per 1,000 words. Each candidate cut sits halfway between two neighbouring values and is scored by the weighted Gini of the two sides.

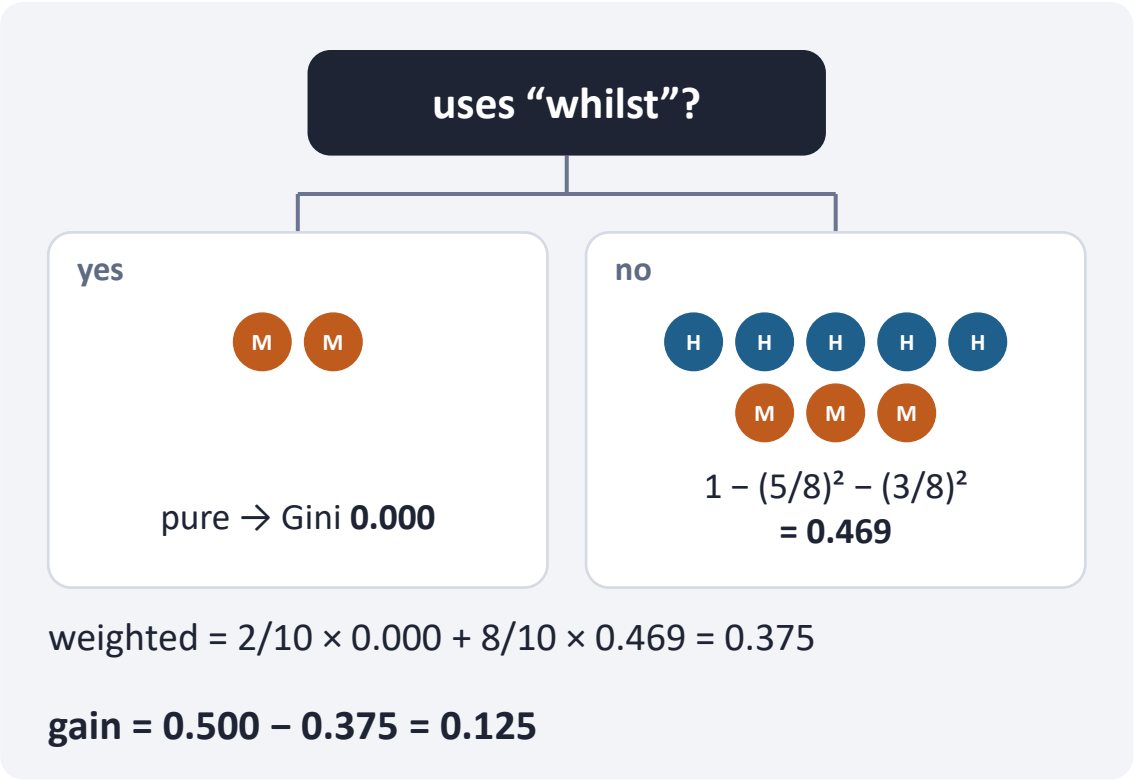


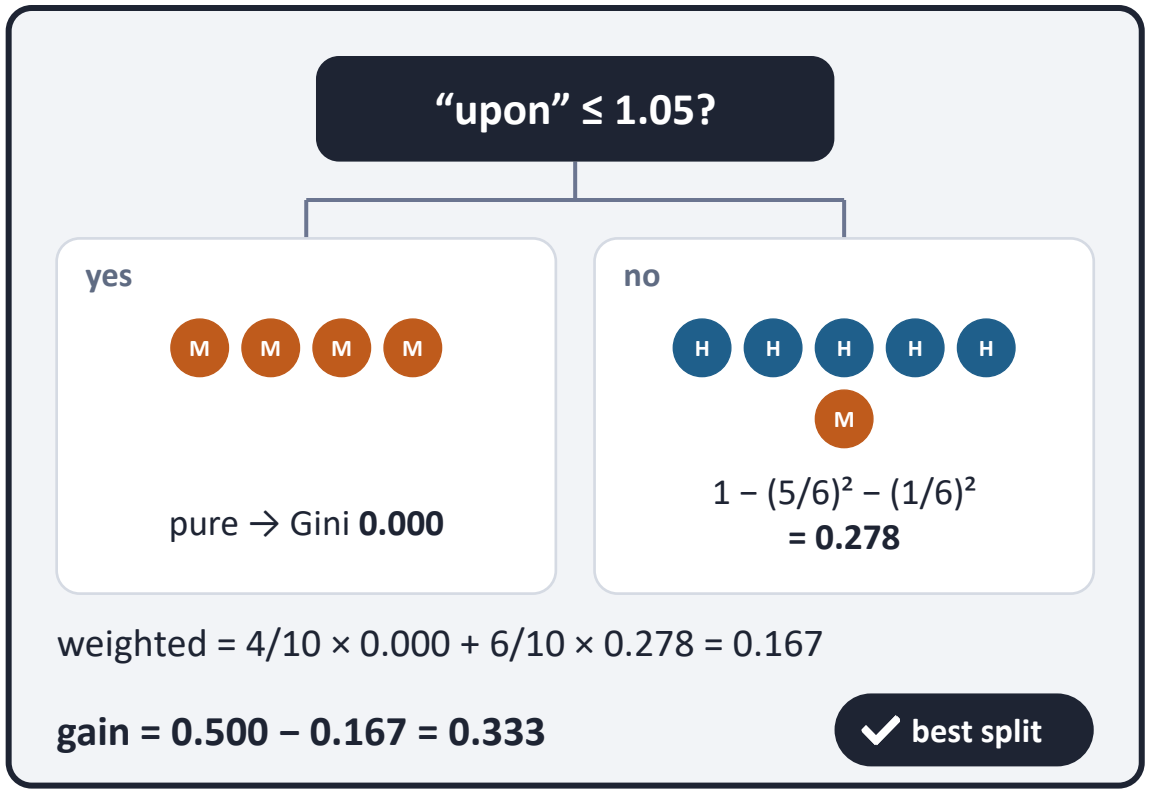
Best cut for “upon”: ≤ 1.05
weighted Gini = 4/10 × 0.000 + 6/10 × 0.278 = 0.167 gain = 0.500 − 0.167 = 0.333

STEP 1 · THE ROOT

Compare features: “upon” wins the root

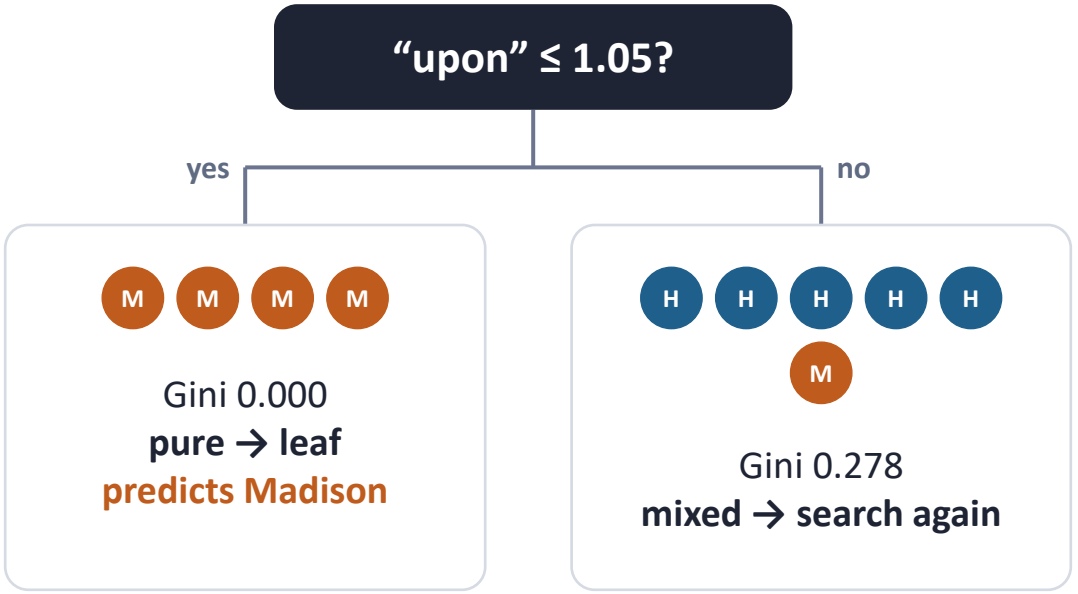
Each feature offers its best split; the algorithm keeps the one with the lowest weighted Gini.





Root question: Is “upon” ≤ 1.05 per 1,000 words?

Repeat inside each branch until it is pure



The same search, inside the “no” branch (6 essays)

uses “whilst”?

M
yes · **0.000**

H H H H H
no · **0.000**

weighted 0.000 · gain 0.278

best split

“upon” ≤ 2.75?

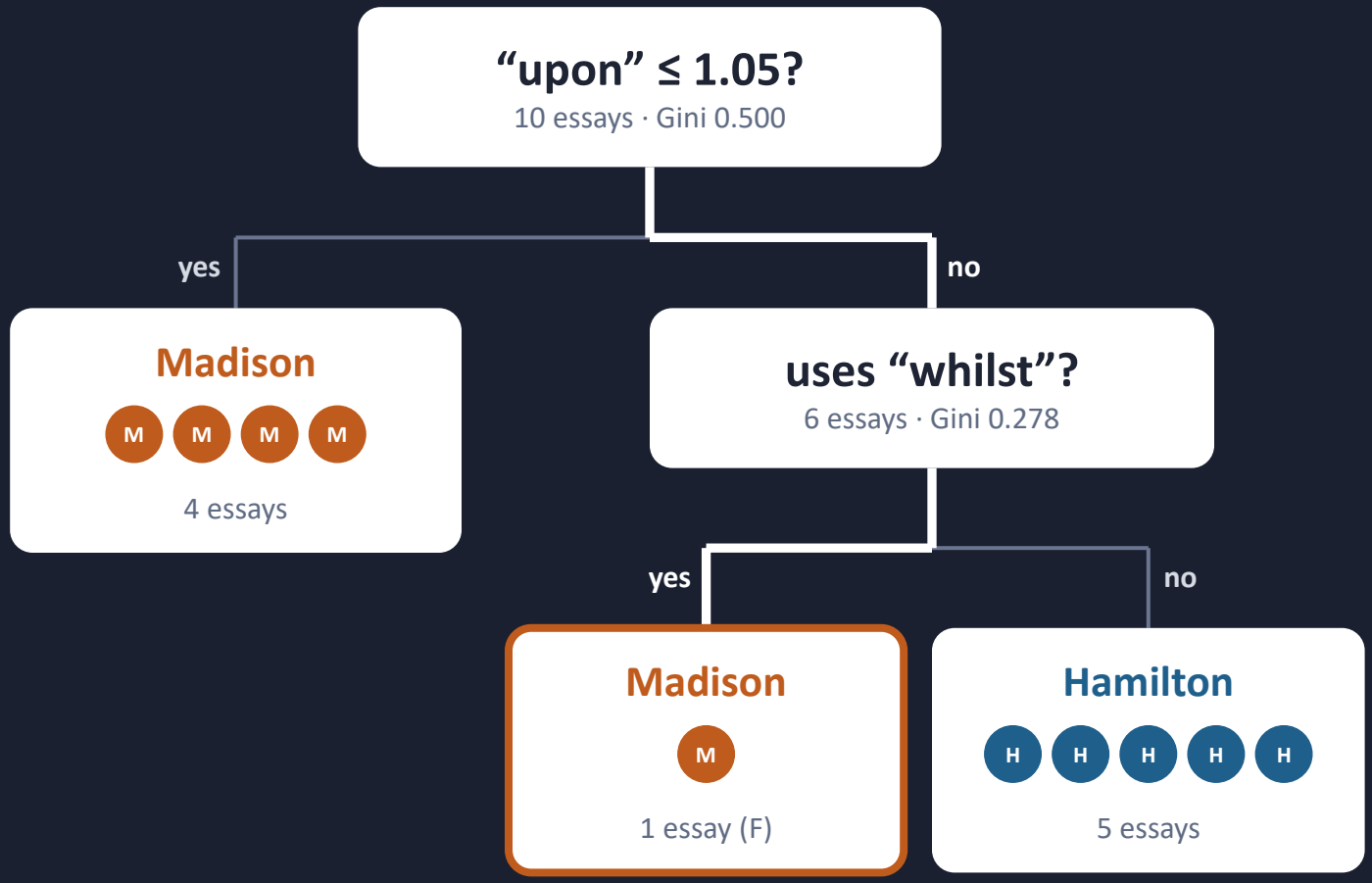
H H
M
yes · **0.444**

H H H
no · **0.000**

weighted 0.222 · gain 0.056 (best of 5 cuts)

When does it stop? Here, as soon as every group is pure: three leaves, done.
On real data we also set limits: `max_depth` (questions per path), `min_samples_leaf` (smallest leaf), `min_impurity_decrease` (smallest gain worth a split).

The finished tree, and a disputed essay



A disputed essay

“upon” = 2.2 per 1,000 words, uses “whilst”

2.2 ≤ 1.05? no → uses “whilst”? yes
→ predicted author: **Madison**

scikit-learn grows the same tree

```
|--- upon <= 1.05
|   |--- class: Madison
|--- upon >  1.05
|   |--- whilst <= 0.50
|   |   |--- class: Hamilton
|   |--- whilst >  0.50
|   |   |--- class: Madison
```

export_text output · whilst coded 0 = no, 1 = yes

⚠ That “whilst” leaf rests on one essay (F): deep trees memorise quirks. Cap the depth, prune, or average many trees (random forests).